

Preliminary Report: Mesh Dependence in PDE-Constrained Optimisation with Firedrake, pyadjoint, and PETSc TAO

Bert Jim Chang
Supervisor: David A. Ham

6 July 2026

1 Introduction and context

Firedrake and its adjoint extension `pyadjoint` support several optimisers: SciPy for general use, ROL (the Rapid Optimisation Library) as the backend for the Fireshape shape-optimisation toolkit, and PETSc TAO (Toolkit for Advanced Optimisation) as a more recent option that recent Firedrake demos recommend for larger or parallel jobs¹. For this to work, TAO has to be shown to behave correctly on the kind of problems Firedrake users solve: problems where the thing being optimised is a function on some domain, not a vector of numbers, and where the right way to measure angle and distance on the control space is a Hilbert-space inner product rather than the standard Euclidean one.

This project aims to provide that empirical evidence, and works out how the `pyadjoint` interface should make the corresponding Riesz-map mechanism available to users. The test problem comes from Schwedes et al. (2017) in their SpringerBrief on mesh-dependence in PDE-constrained optimisation. Its Table 2.2 makes this pattern clear: an optimiser hard-coded to the Euclidean ℓ^2 inner product produces iteration counts that grow polynomially with mesh grading, while one that uses the natural L^2 inner product stays essentially flat. The book demonstrates this with FEniCS and Moola. This project reproduces the same result with Firedrake, `pyadjoint`, and PETSc TAO, checks whether TAO can be made to match the flat Moola row, and extends the argument one level deeper into the inner Krylov solve of Newton’s method.

2 Background and literature

The main reference text is Schwedes et al. (2017). Its main result is that gradient-based optimisation over a Hilbert function space is mesh-independent if and only if the algorithm respects that Hilbert space’s inner product. Chapter 1 gives the main foundational tools we need: the Riesz representation theorem, Fréchet derivatives, adjoint equations, and function-space versions of steepest descent, Newton-CG, and BFGS. Chapter 2 tests this claim experimentally, using the Table 2.2 setup introduced above.

The underlying object is the Riesz map, the operator that turns a functional’s derivative (which naturally sits in the dual space H^*) into an element of H that the optimiser can use as a search direction. In the discrete L^2 setting, this map is $\mathcal{R}_{L^2} = M_h^{-1}$, the inverse of the assembled Galerkin mass matrix (Schwedes et al. (2017, Eq. 1.125)). Applying M_h^{-1} converts the raw coefficient gradient into its L^2 primal form, and skipping it leaves the mesh-scaling factor in M_h uncanceled, which produces the polynomial growth on graded meshes (Hinze et al. (2009)).

The same argument applies to preconditioning Krylov solvers, one level deeper than the outer optimisation. Kirby (2010) shows that discrete finite element operators are mesh-independently conditioned when preconditioned with the Riesz map of the correct Hilbert space. Schwedes cites this as a pointer to the Krylov version of the same idea (§1.3.3 and §2.1) but does not run an inner-Krylov experiment: Chapter 2’s experiments are all outer L-BFGS. Newton’s method has an outer Newton loop and an

¹See e.g. the Firedrake full-waveform inversion demo, https://www.firedrakeproject.org/demos/full_waveform_inversion.py.html.

inner Krylov solve on the reduced Hessian, so the same reasoning predicts the inner solve inherits its own mesh-dependence unless preconditioned with M_h^{-1} . This project fills that gap on Schwedes' Poisson test problem.

3 Objectives

The overall goal is to give Firedrake the empirical evidence and the interface design it needs to use PETSc TAO more widely as an optimisation backend. This splits into three main objectives.

1. **Show that TAO works correctly at the outer level.** Reproduce panel (a) of Table 2.2 in Schwedes et al. (2017), this time in the Firedrake and pyadjoint stack. Show that the SciPy row grows with the mesh ratio and that our own hand-written Hilbert-space L-BFGS row stays flat. Find the settings that make TAO's `lmvm` solver match the flat row, and try the other TAO solvers (`nls`, `bqnls`, `cg`) to see which ones behave well and which do not. This gives Firedrake a clear answer to the question "does TAO handle function-space optimisation correctly, and under what settings?".
2. **Fix the mesh-dependence at the inner solver level.** TAO's `nls` solver runs Newton's method, which takes two outer iterations on this problem because the Poisson problem is quadratic. Each Newton step then solves a linear system inside itself, and that inner solve has its own iteration count that the outer table does not show. Show that this inner count grows with mesh grading, and that adding an L^2 Riesz-map preconditioner to the inner solve fixes it. The book does not test this level, so this is the main novel piece of the project.
3. **Work out the pyadjoint interface.** What should pyadjoint's `TAOSolver` wrapper actually do? Three options: (a) apply the Riesz map automatically when the solver is built, (b) make the user ask for it explicitly with `Control(m, riesz_map="L2")`, or (c) some hybrid of the two. Each option has pros and cons, especially when a user wants to pass in their own preconditioner. Determine which option makes the most sense and why.

4 Methods

The model problem is the Poisson optimal control problem from Schwedes et al. (2017, Sect. 1.6), in which we look for a source term $m \in L^2(\Omega)$ that minimises

$$\tilde{J}(m) = \frac{1}{2} \|u - d\|_{L^2(\Omega)}^2 + \frac{\alpha}{2} \|m\|_{L^2(\Omega)}^2 \quad \text{subject to} \quad -\Delta u = m \text{ on } \Omega, \quad u = 0 \text{ on } \partial\Omega.$$

We derive the reduced gradient by hand using the adjoint method, and compute it at run time using pyadjoint, which records a tape of the forward solve and plays it back in reverse. Both the forward and adjoint solves use P_1 continuous Lagrange finite elements. We verify the implementation two ways: a manufactured-solution convergence test, showing second-order convergence in the $L^2(\Omega)$ error, and a Taylor test on the reduced gradient, showing second-order decay of the first-order remainder. After this, the scalar viscous Burgers optimal control problem is the natural next step to test.

We build graded meshes on the unit square using Netgen (Schöberl, 1997) at target ratios $r \in \{4, 8, 16, 32, 64, 128\}$, following the method in Schwedes et al. (2017, Sect. 2.3). The realised ratios of $h_{\max}/h_{\min}$, measured from Firedrake's `CellDiameter` field, come out at roughly 1.5 to 2 times the target.

For objective 1 we run two outer optimisers side by side. The first is a hand-written Hilbert-space L-BFGS following Schwedes et al. (2017, Sect. 1.5.4.1), written in Python and fully transparent, which acts as our reference. The second is PETSc TAO through pyadjoint's `TAOSolver` wrapper, with `Control(m, riesz_map="L2")` using M_h as the LMVM initial-Hessian seed. For objective 2 we write a plain PETSc Python PC (`RieszMapPCContext`), following the preconditioner pattern in the

Firedrake saddle-point systems demo². It assembles the mass matrix, factorises it once by LU, and applies M_h^{-1} on every inner KSP call, attached programmatically after `TAOSolver` is built.

5 Preliminary results

Panel (a) reproduction. On four Netgen graded meshes at target ratios $r \in \{4, 16, 64, 128\}$, our hand-written Hilbert-space L-BFGS converged to $\|\mathcal{R}_{L^2}(J')\|_{L^2} \leq 10^{-7}$ in 8, 7, 8, and 9 iterations. The SciPy L-BFGS-B row (run in ℓ^2 , with the convergence check done externally in L^2) produced counts of 46, 195, and 786 for $r \in \{4, 16, 64\}$, growing roughly linearly with the mesh grading. TAO’s `lmvm` solver reproduced the flat row once we raised the default history length of five up to 100. At the default value, the rank-five quasi-Newton update did not capture enough of the discrete-Hessian spectrum. Across the wider TAO family, the Newton-class solvers and `lmvm-at-100` were mesh-independent, whereas optimisers `bqnls` and `cg` were not.

Inner-CG mesh dependence and its fix. TAO’s `nls` solver takes two outer Newton iterations regardless of the mesh, because the Poisson problem is quadratic. Each Newton step, however, solves a linear system using an inner CG, and the outer table does not show that inner count. On the same mesh sweep, the total inner-CG count grew from 56 at $r = 4$ to 807 at $r = 128$. When we installed the L^2 Riesz-map preconditioner on TAO’s inner KSP, the count stayed flat at 15 across the whole sweep. This is the panel (a) result carried down to the inner Krylov solve; it is written up as §10.8 of the dissertation.

6 Work plan for the remaining time

- **Diagnose `bqnls`’s line search.** TAO’s `bqnls` solver grows with the mesh at its default history of five. Two things could be going wrong: it could be the same history-size issue that stock `lmvm` had, or the projected line search could be doing something wrong. The plan is to turn on `tao_ls_monitor` and rerun the mesh sweep. The output should distinguish between the two.
- **H^1 Riesz-map preconditioner sweep.** The preconditioner module already handles the H^1 form. Adding an H^1 column to the inner-CG table would round off the story symmetrically with panels (a) and (b) of Table 2.2.
- **Write up the interface design.** Draft a short document that lays out how the pyadjoint `TAOSolver` wrapper should expose the Riesz-map mechanism to users. This is the deliverable for objective 3, and it is what would help Firedrake use TAO more widely.

References

- Michael Hinze, René Pinnau, Michael Ulbrich, and Stefan Ulbrich. *Optimization with PDE Constraints*, volume 23 of *Mathematical Modelling: Theory and Applications*. Springer, 2009.
- Robert C. Kirby. From functional analysis to iterative methods. *SIAM Review*, 52(2):269–293, 2010. doi: 10.1137/070706914.
- Joachim Schöberl. NETGEN: An advancing front 2D/3D-mesh generator based on abstract rules. *Computing and Visualization in Science*, 1(1):41–52, 1997.
- Tobias Schwedes, David A. Ham, Simon W. Funke, and Matthew D. Piggott. *Mesh Dependence in PDE-Constrained Optimisation: An Application in Tidal Turbine Array Layouts*. SpringerBriefs in Mathematics of Planet Earth. Springer, 2017. doi: 10.1007/978-3-319-59483-5.

²https://www.firedrakeproject.org/demos/saddle_point_systems.py.html.